



Instrumentation d'une cuve de fermentation de vin

ELEC 424 – Actionneurs

Spring 2025-2026

Prepared By

Jad ABDALLAH
Elissa KASTOUN
Lea EID

Presented To

Dr. Rouba ALRAMMOUZ

Table des matières

1	Introduction	3
2	Cahier des charges	4
2.1	Type de mouvement	4
2.2	Environnement de fonctionnement	4
2.3	Fréquence de manœuvre	4
2.4	Exigence de sécurité	4
3	Étude comparative des actionneurs	5
3.1	Actionneur pneumatique	5
3.2	Actionneur électrique	5
3.3	Tableau comparatif	6
3.4	Choix de la solution	6
4	Étude du marché	7
4.1	Composants industriels possibles	7
4.2	Composants pour prototype Arduino	8
5	Architecture matérielle	9
5.1	Architecture générale du système	9
5.2	Entrées du système	9
5.3	Sorties du système	9
5.4	Stratégie des deux capteurs de niveau	10
5.5	Mesure de température par thermistance NTC	10
5.5.1	Pont diviseur de tension	10
5.5.2	Modèle Beta (équation de Steinhart simplifiée)	10
5.5.3	Calibration du composant	11
5.5.4	Exemple de calcul	11
5.5.5	Précision et amélioration	11
5.6	Branchement proposé pour le prototype	12
6	Architecture logicielle	13
6.1	Principe de fonctionnement	13
6.2	États du système	13
6.3	Conditions d'alarme	13
6.4	Affichage LCD	14
6.5	Organigramme logique	14
7	Bilan énergétique	15
7.1	Hypothèses du prototype	15
7.2	Puissance électrique	15
7.3	Énergie par cycle	15
8	Résultats attendus	16

9	Prototype réalisé	17
10	Limitations identifiées	19
11	Discussion	20
12	Conclusion	21
13	Améliorations possibles	22
	Références	23

1. Introduction

La fermentation du vin est un procédé biologique durant lequel les levures transforment les sucres présents dans le moût de raisin en alcool et en dioxyde de carbone. Ce procédé se déroule dans une cuve de fermentation, où plusieurs paramètres doivent être surveillés afin d'assurer la qualité du produit final et la sécurité de l'installation.

Dans ce projet, on s'intéresse à l'instrumentation d'une cuve de fermentation et plus précisément à l'automatisation de l'ouverture et de la fermeture d'une vanne principale de régulation de débit. Conformément au cahier des charges, cette vanne suit un cycle temporisé (manœuvre toutes les 5 minutes), et les capteurs assurent la couche de sécurité qui impose une fermeture d'urgence en cas d'anomalie.

L'objectif principal est de choisir un actionneur adapté pour commander une vanne papillon fonctionnant par rotation d'un quart de tour. Deux solutions sont étudiées :

- un actionneur pneumatique alimenté par air comprimé ;
- un actionneur électrique basé sur un moteur rotatif avec réducteur.

Le système proposé doit être adapté à un environnement humide, assurer une fréquence de manœuvre élevée, et respecter une exigence importante de sécurité : la vanne doit pouvoir se fermer en urgence en cas d'anomalie.

2. Cahier des charges

Le cahier des charges définit les contraintes fonctionnelles et techniques du système à concevoir.

2.1 Type de mouvement

La vanne étudiée est une vanne papillon. Ce type de vanne fonctionne par une rotation de 90 degrés (quart de tour). L'actionneur doit donc être capable de faire tourner l'axe de la vanne entre deux positions principales :

- position fermée : angle de 0° ;
- position ouverte : angle de 90° .

2.2 Environnement de fonctionnement

La cuve de fermentation se trouve dans un environnement humide avec présence possible de projections de liquide, de vapeur ou de condensation. La température peut varier entre 15°C et 30°C . Les éléments électriques doivent être placés dans un boîtier de protection de type IP65 ou IP67.

2.3 Fréquence de manœuvre

La vanne doit pouvoir s'ouvrir et se fermer toutes les 5 minutes, soit environ 12 cycles par heure :

$$N_{\text{cycles}} = 12 \times 24 = 288 \text{ cycles/jour} \quad (2.1)$$

2.4 Exigence de sécurité

La vanne doit pouvoir se fermer en cas de situation anormale :

- coupure de courant ;
- panne d'air comprimé ;
- pression trop élevée ;
- température anormale ;
- activation d'un bouton d'arrêt d'urgence.

Cette exigence correspond à une logique de **sécurité positive** : la position de sécurité (fermée) doit être atteinte par *perte* d'énergie et non par commande. C'est un point déterminant pour le choix de l'actionneur (voir chapitre suivant) : seule une solution à rappel mécanique (ressort) garantit la fermeture lors d'une coupure de courant, car dans ce cas le microcontrôleur n'est plus alimenté et ne peut exécuter aucune commande.

3. Étude comparative des actionneurs

3.1 Actionneur pneumatique

Un actionneur pneumatique utilise de l'air comprimé pour produire un mouvement mécanique. Pour une vanne papillon, on utilise généralement un actionneur pneumatique quart de tour à simple effet avec ressort de rappel. En cas de coupure d'alimentation ou de panne, le ressort ramène automatiquement la vanne en position fermée, répondant ainsi naturellement à l'exigence de sécurité positive.

Avantages principaux :

- très adapté aux environnements industriels humides ;
- couple élevé disponible ;
- sécurité positive native avec ressort de rappel ;
- grande durée de vie pour les manœuvres fréquentes.

Inconvénients principaux :

- nécessite une source d'air comprimé et une électrovanne ;
- coût élevé pour un prototype étudiant ;
- maintenance du circuit pneumatique nécessaire.

3.2 Actionneur électrique

Un actionneur électrique utilise un moteur pour produire le mouvement de rotation. Pour le prototype, un servomoteur SG90 est utilisé car il permet de commander directement un angle entre 0° et 90° via un signal PWM de l'Arduino.

Avantages principaux :

- facile à commander avec Arduino via signal PWM ;
- pas besoin d'air comprimé ;
- coût très réduit à petite échelle ;
- contrôle précis de la position.

Inconvénients principaux :

- sécurité positive difficile à obtenir : un servomoteur standard reste figé dans sa dernière position lors d'une coupure de courant (il ne se referme pas seul) ;
- protection nécessaire contre l'humidité ;
- couple limité pour les petits servomoteurs.

3.3 Tableau comparatif

TABLE 3.1 – Comparaison entre actionneur pneumatique et actionneur électrique.

Critère	Pneumatique	Électrique (servo)
Mouvement requis	Très adapté au quart de tour	Adapté avec moteur et réducteur
Couple disponible	Élevé	Variable selon le moteur
Commande	Électrovanne + relais	Signal PWM direct Arduino
Sécurité positive	Native (ressort de rappel)	Non native (figé sur coupure)
Environnement humide	Très adapté	Nécessite protection IP
Maintenance	Réseau d'air	Limitée
Coût prototype	Élevé	Faible
Fiabilité industrielle	Très élevée	Bonne si actionneur protégé
Complexité installation	Moyenne à élevée	Faible à moyenne

3.4 Choix de la solution

Pour une application industrielle réelle, la solution la plus adaptée est l'**actionneur pneumatique quart de tour à simple effet avec ressort de rappel**. Ce choix est justifié principalement par l'exigence de sécurité positive : en cas de coupure électrique, le ressort ferme automatiquement la vanne, ce qu'aucune logique logicielle ne peut garantir puisque le microcontrôleur n'est alors plus alimenté.

Pour le prototype, la solution retenue est donc double :

- **Solution industrielle recommandée** : actionneur pneumatique quart de tour avec ressort de rappel ;
- **Prototype réalisé** : servomoteur SG90 commandé par Arduino Uno.

4. Étude du marché

4.1 Composants industriels possibles

TABLE 4.1 – Exemples de composants industriels pour le système réel.

No.	Composant	Fonction	Prix estimé
1	Actionneur pneumatique quart de tour	Rotation de la vanne	80–250 USD
2	Électrovanne pneumatique	Commande de l'air comprimé	20–80 USD
3	Vanne papillon inox	Régulation du débit	50–200 USD
4	Capteur de pression	Détection de surpression	15–80 USD
5	Capteur de température PT100	Surveillance thermique	10–50 USD
6	Capteur de niveau ultrasonique	Mesure du niveau de liquide	5–20 USD
7	Boîtier IP65/IP67	Protection électrique	15–60 USD
8	Alimentation 24V DC	Alimentation du système	15–50 USD

4.2 Composants pour prototype Arduino

TABLE 4.2 – Composants utilisés pour le prototype Arduino.

No.	Composant	Fonction	Prix estimé
1	Arduino Uno	Traitement des données et commande	10–25 USD
2	Servomoteur SG90	Simulation de la vanne (0°–90°)	3–12 USD
3	Thermistance NTC (non marquée) + résistance 10 k Ω	Mesure de température (pont diviseur)	1–3 USD
4	Capteur ultrasonique HC-SR04	Mesure du niveau de liquide	2–5 USD
5	Module niveau d'eau HW-038	Détection de débordement	1–3 USD
6	Potentiomètre 10k Ω	Simulation de pression	1–2 USD
7	Bouton poussoir	Arrêt d'urgence manuel	1 USD
8	LED rouge et verte	Indication d'état (sain / défaut)	1 USD
9	Buzzer	Signal d'alarme sonore	1–3 USD
10	Écran LCD 16x2 avec module I2C	Affichage état système en temps réel	3–8 USD
11	Câblage / breadboard	Connexions électriques	3–5 USD

5. Architecture matérielle

5.1 Architecture générale du système

Le système est composé de trois parties : capteurs, traitement, et actionnement. Les capteurs mesurent les grandeurs physiques du procédé. L'Arduino compare les valeurs aux seuils de sécurité et commande l'actionneur. En cas d'anomalie, la vanne est immédiatement fermée.

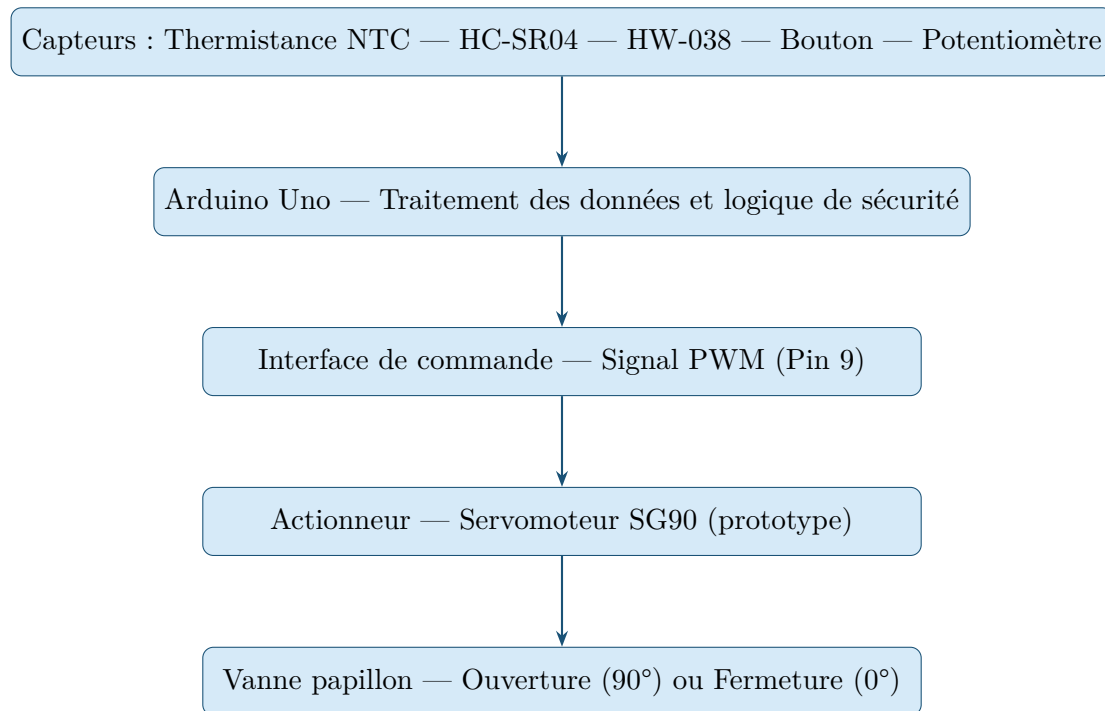


FIGURE 5.1 – Architecture matérielle générale du système.

5.2 Entrées du système

- température — thermistance NTC montée en pont diviseur (lecture analogique sur A2) ;
- niveau de liquide par distance ultrasonique (HC-SR04) ;
- détection de débordement (HW-038) ;
- pression simulée par potentiomètre 10k Ω ;
- bouton d'arrêt d'urgence.

5.3 Sorties du système

- commande du servomoteur SG90 (signal PWM) ;
- LED verte — système sain (fonctionnement normal) ;
- LED rouge — défaut / état d'urgence ;
- buzzer — alarme sonore (actif uniquement en défaut) ;
- écran LCD 16 $\times$ 2 (I2C) — affichage de l'état, des valeurs capteurs, de la position de la vanne et de la cause du défaut.

5.4 Stratégie des deux capteurs de niveau

Le système comporte deux capteurs de niveau de technologies différentes, avec des rôles *volontairement* distincts (redondance par diversité) :

- le **HC-SR04** fournit la mesure *continue* du niveau (distance à la surface), affichée à l'écran, et déclenche le seuil « cuve pleine » ;
- le **module résistif HW-038** agit comme *interrupteur de débordement* indépendant.

Ainsi, une erreur de mesure de l'ultrason (écho parasite, mousse, surface agitée) est rattrapée par une sonde de principe physique différent. Pour limiter la corrosion, le module HW-038 n'est pas alimenté en permanence : sa borne d'alimentation est pilotée par une broche numérique (A3) et n'est mise sous tension que le temps de la mesure.

5.5 Mesure de température par thermistance NTC

La température est mesurée à l'aide d'une thermistance NTC (*Negative Temperature Coefficient*), dont la résistance diminue lorsque la température augmente. Le composant utilisé n'étant pas marqué (aucune référence fabricant), il a été **caractérisé expérimentalement** plutôt que d'après une fiche technique.

5.5.1 Pont diviseur de tension

Une thermistance étant un composant passif, elle est insérée dans un pont diviseur avec une résistance fixe $R_s = 10\text{ k}\Omega$, la résistance fixe étant placée côté 5 V :

$$5\text{V} \longrightarrow R_s \longrightarrow \underbrace{\text{noeud (A2)}}_{V_{A2}} \longrightarrow R_{ntc} \longrightarrow \text{GND}$$

La tension lue sur la broche A2 vaut :

$$V_{A2} = V_{cc} \frac{R_{ntc}}{R_s + R_{ntc}} \quad (5.1)$$

Le convertisseur analogique-numérique 10 bits de l'Arduino renvoie une valeur $n \in [0, 1023]$ telle que :

$$n = 1023 \frac{R_{ntc}}{R_s + R_{ntc}} \quad (5.2)$$

d'où l'on extrait la résistance de la thermistance :

$$\boxed{R_{ntc} = R_s \frac{n}{1023 - n}} \quad (5.3)$$

5.5.2 Modèle Beta (équation de Steinhart simplifiée)

La relation résistance-température d'une thermistance est modélisée par l'équation Beta :

$$\frac{1}{T} = \frac{1}{T_0} + \frac{1}{B} \ln\left(\frac{R_{ntc}}{R_0}\right) \quad (T \text{ en kelvin}) \quad (5.4)$$

La température en degrés Celsius est ensuite :

$$T(^{\circ}\text{C}) = T(\text{K}) - 273,15 \quad (5.5)$$

5.5.3 Calibration du composant

Le composant a été mesuré directement au multimètre : $R_0 = 8700 \, \Omega$ à $T_0 = 28 \, ^\circ\text{C} = 301,15 \, \text{K}$ (température ambiante confirmée lors de l'essai). Le coefficient Beta n'étant pas connu, la valeur typique des thermistances $10 \, \text{k}\Omega$ du commerce a été adoptée : $B = 3950 \, \text{K}$.

Ancrer le calcul sur la mesure réelle (R_0, T_0) plutôt que sur une valeur nominale supposée présente un avantage : cela **annule l'incertitude sur la résistance nominale** du composant (le fait qu'il soit « 10k » ou non n'importe plus). Seule subsiste l'incertitude sur B .

5.5.4 Exemple de calcul

Supposons une lecture $n = 567$. La résistance vaut :

$$R_{ntc} = 10000 \times \frac{567}{1023 - 567} = 10000 \times \frac{567}{456} \approx 12434 \, \Omega \quad (5.6)$$

En injectant dans l'équation Beta :

$$\frac{1}{T} = \frac{1}{301,15} + \frac{1}{3950} \ln\left(\frac{12434}{8700}\right) \approx 3,4110 \times 10^{-3} \, \text{K}^{-1} \quad (5.7)$$

$$T \approx 293,2 \, \text{K} \approx 20,0 \, ^\circ\text{C} \quad (5.8)$$

5.5.5 Précision et amélioration

Avec un seul point de mesure et un Beta supposé, la précision est bonne au voisinage de $28 \, ^\circ\text{C}$ mais se dégrade aux extrêmes de la plage. Pour la fenêtre d'alarme $15\text{--}30 \, ^\circ\text{C}$, cette précision est suffisante. Une amélioration consiste à effectuer une **calibration à deux points** (par exemple un bain de glace à $0 \, ^\circ\text{C}$), ce qui permet de *calculer* le Beta réel du composant :

$$B = \frac{\ln(R_1/R_2)}{\frac{1}{T_1} - \frac{1}{T_2}} \quad (5.9)$$

Remarque pratique : la valeur réelle de la résistance fixe « 10k » doit être mesurée au multimètre et reportée dans le code (R_s), car elle intervient directement dans le calcul.

5.6 Branchement proposé pour le prototype

TABLE 5.1 – Affectation des broches Arduino pour le prototype.

Composant	Broche Arduino	Remarque
Servo SG90 (signal)	D9 (PWM)	0°=fermé, 90°=ouvert
Thermistance NTC	A2	Pont diviseur 10k Ω , équation Beta
HC-SR04 Trig	D10	Impulsion 10 μ s
HC-SR04 Echo	D7	pulseIn() avec timeout 30ms
LED verte (système sain)	D6	Résistance 220 Ω en série
LED rouge (défaut)	D5	Résistance 220 Ω en série
Buzzer	D8	Alarme sonore (actif en défaut)
Bouton urgence	D2 (INT0)	Interruption matérielle FALLING
Module HW-038 (S)	A0	Seuil calibré à 370/1023
Alim. module HW-038 (+)	A3	Alimentation pulsée (anti-corrosion)
Potentiomètre (curseur)	A1	Pression simulée, seuil 2,5 bar ($\approx 853/1023$)
LCD I2C (SDA)	A4	Bibliothèque LiquidCrystal_I2C, adresse 0x27
LCD I2C (SCL)	A5	Horloge I2C

*Note : le servomoteur est alimenté par une **source 5V externe séparée**, avec masse commune à l'Arduino et un condensateur de découplage ($\approx 470\mu F$) sur son rail d'alimentation. Ceci évite que l'appel de courant du servo ne provoque une chute de tension sur l'Arduino (redémarrage intempestif, perturbation de l'écran LCD).*

6. Architecture logicielle

6.1 Principe de fonctionnement

Le programme Arduino suit une machine à états finis (FSM) entièrement **non bloquante** : la cadence de lecture des capteurs, l’affichage et le buzzer sont gérés par `millis()`. À chaque itération, le microcontrôleur lit tous les capteurs et vérifie les conditions de sécurité. La priorité est toujours donnée à la sécurité : toute anomalie provoque immédiatement le passage à l’état **EMERGENCY**.

Note : la période de manœuvre spécifiée est de 5 minutes (cahier des charges). Pour la démonstration sur maquette, elle est réduite à 10 secondes (`DUREE_CYCLE_MS`) afin d’observer plusieurs cycles rapidement ; le passage en production se fait en remplaçant cette seule constante.

6.2 États du système

TABLE 6.1 – États logiques du système (FSM).

État	Description	Transition sortante
IDLE	Initialisation, vanne fermée	Toujours → OPEN
OPEN	Vanne ouverte à 90°	Fin de cycle → CLOSE / Anomalie → EMERGENCY
CLOSE	Vanne fermée à 0°	Fin de cycle → OPEN / Anomalie → EMERGENCY
EMERGENCY	Vanne fermée, LED rouge fixe, buzzer pulsé	Verrouillé ; acquittement par bouton <i>une fois la cause levée</i> → IDLE

L’état **EMERGENCY** est **verrouillé** (*latched*) : abaisser la grandeur sous son seuil ne suffit pas à sortir de l’alarme. Le retour au fonctionnement normal exige deux conditions simultanées : la disparition de la cause *et* un appui sur le bouton d’acquiescement. Ce comportement est conforme à la logique de sécurité positive : une alarme ne doit pas s’effacer d’elle-même sans intervention de l’opérateur. L’indication d’état est portée par les LED : verte = système sain, rouge = défaut.

6.3 Conditions d’alarme

Le système passe en état **EMERGENCY** si au moins une des conditions suivantes est vraie :

$$\text{Alarme} = \text{Urgence} \vee (T > T_{\max}) \vee (T < T_{\min}) \vee (P > P_{\max}) \vee (\text{Eau} > S_{\text{eau}}) \vee (d < d_{\min}) \quad (6.1)$$

Avec :

- $T_{\min} = 15^{\circ}\text{C}$, $T_{\max} = 30^{\circ}\text{C}$ (plage de fermentation) ;
- $P_{\max} = 2,5 \text{ bar}$ (pression simulée par potentiomètre, échelle 0–3 bar) ;

- $S_{\text{eau}} = 370/1023$ (seuil calibré empiriquement — maximum mesuré à immersion complète : $450/1023$);
- $d_{\text{min}} = 5$ cm (distance ultrasonique — cuve quasi-pleine).

6.4 Affichage LCD

L'écran affiche en permanence l'état du système et la position de la vanne :

- fonctionnement normal — ligne 1 : V:OUVERT (ou FERMEE) ; ligne 2 : valeurs en temps réel T...C P...bar D...cm ;
- défaut — ligne 1 : ALARME V:FERMEE ; ligne 2 : la *cause* du défaut (p.ex. Pression haute, Temp trop haute, Debordement eau, Cuve pleine, Bouton urgence).

6.5 Organigramme logique

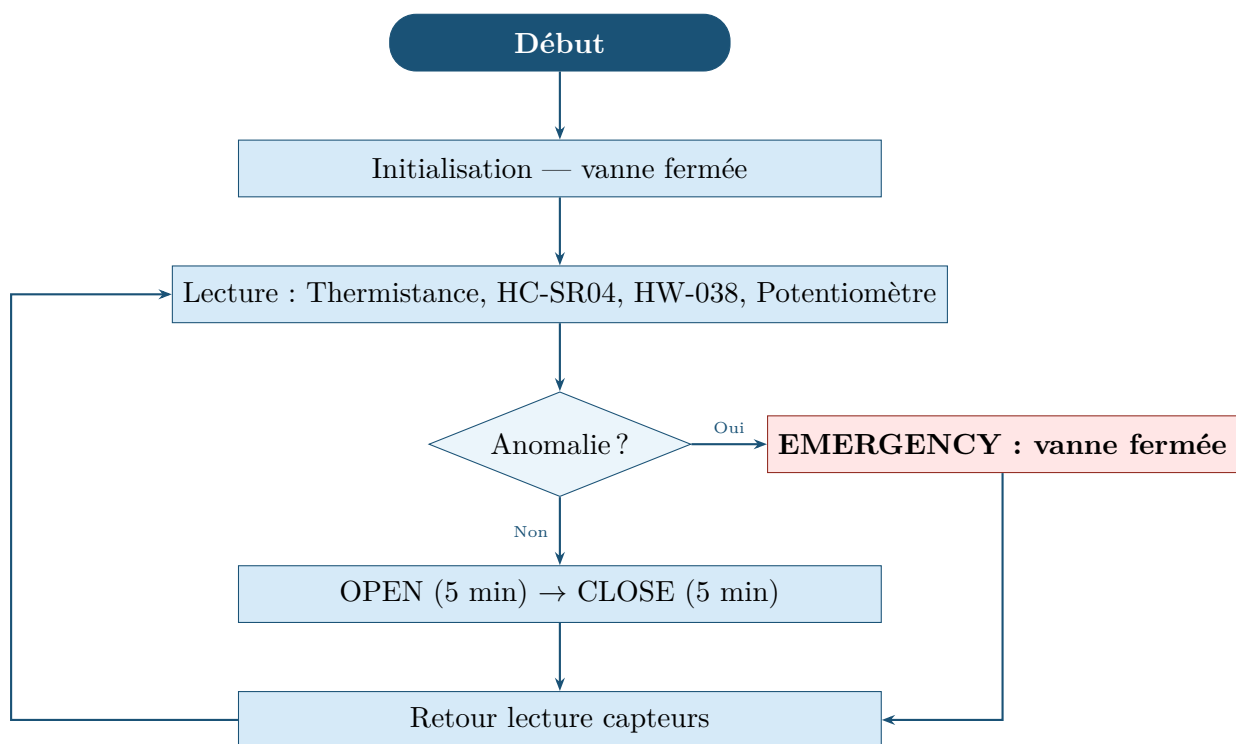


FIGURE 6.1 – Organigramme simplifié du programme Arduino. La période de 5 min du cahier des charges est réduite à 10s sur la maquette pour la démonstration.

7. Bilan énergétique

7.1 Hypothèses du prototype

TABLE 7.1 – Hypothèses de consommation électrique du prototype.

Composant	Tension (V)	Courant typ. (mA)	Puissance (mW)
Arduino Uno (actif)	5V	50	250
Servo SG90 (mouvement)	5V	~250	~1250
Servo SG90 (maintien)	5V	~10	~50
Thermistance NTC (pont)	5V	~0.5	~2.5
HC-SR04	5V	15	75
HW-038	5V	<20	<100
LEDs (2×)	5V	20 chacune	200
Buzzer	5V	30	150
Potentiomètre 10kΩ	5V	~0.5	~2.5

7.2 Puissance électrique

$$P = U \cdot I \quad (7.1)$$

Puissance maximale (servo en mouvement) :

$$P_{\text{total}} \approx 250 + 1250 + 75 + 100 + 200 + 150 + 2.5 + 2.5 \approx 2030 \text{ mW} \quad (7.2)$$

Puissance typique (servo en maintien) :

$$P_{\text{typique}} \approx 250 + 50 + 75 + 100 + 200 + 2.5 + 2.5 \approx 680 \text{ mW} \quad (7.3)$$

7.3 Énergie par cycle

Si le servomoteur fonctionne 2 secondes par manœuvre, un cycle complet (ouverture + fermeture) représente 4 secondes de mouvement actif :

$$E_{\text{cycle}} = P_{\text{servo}} \cdot t = 1.25 \times 4 = 5 \text{ J} \quad (7.4)$$

Avec 288 cycles par jour :

$$E_{\text{jour}} = 288 \times 5 = 1440 \text{ J} \quad (7.5)$$

$$E_{\text{jour}} = \frac{1440}{3600} \approx 0.4 \text{ Wh} \quad (7.6)$$

Cette consommation est très faible pour le prototype. Dans un système industriel réel, la consommation principale proviendrait de l'actionneur pneumatique et du compresseur.

8. Résultats attendus

TABLE 8.1 – Réactions attendues du système selon les situations.

Situation	Action du système	Indication
Fonctionnement normal	Ouverture/fermeture cyclique (5 min)	LED verte
Température < 15°C ou > 30°C	Fermeture immédiate (verrouillée)	LED rouge + buzzer
Pression simulée > 2,5 bar	Fermeture immédiate (verrouillée)	LED rouge + buzzer
Niveau eau > 370/1023	Fermeture immédiate (verrouillée)	LED rouge + buzzer
Distance < 5 cm	Fermeture immédiate (verrouillée)	LED rouge + buzzer
Bouton urgence pressé	Fermeture immédiate (verrouillée)	LED rouge + buzzer
Acquittement (cause levée + bouton)	Retour au fonctionnement normal	LED verte

9. Prototype réalisé

Le prototype complet a été assemblé et testé. La maquette comprend la cuve (récipient transparent), les capteurs immergés ou montés en hauteur, le panneau de commande (LCD, bouton d'urgence, potentiomètre, LED, buzzer) et l'Arduino.

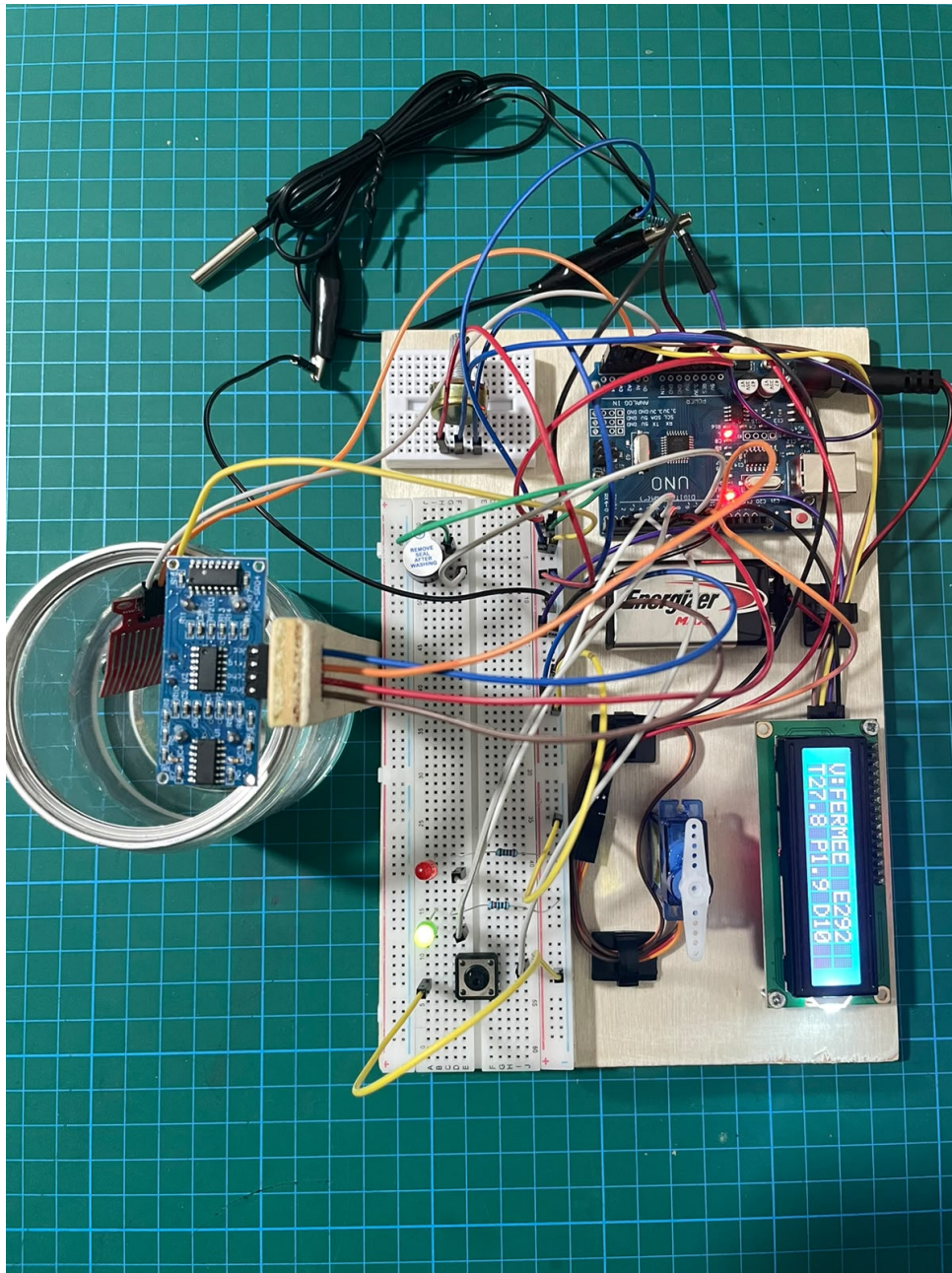


FIGURE 9.1 – Vue d'ensemble du prototype assemblé.

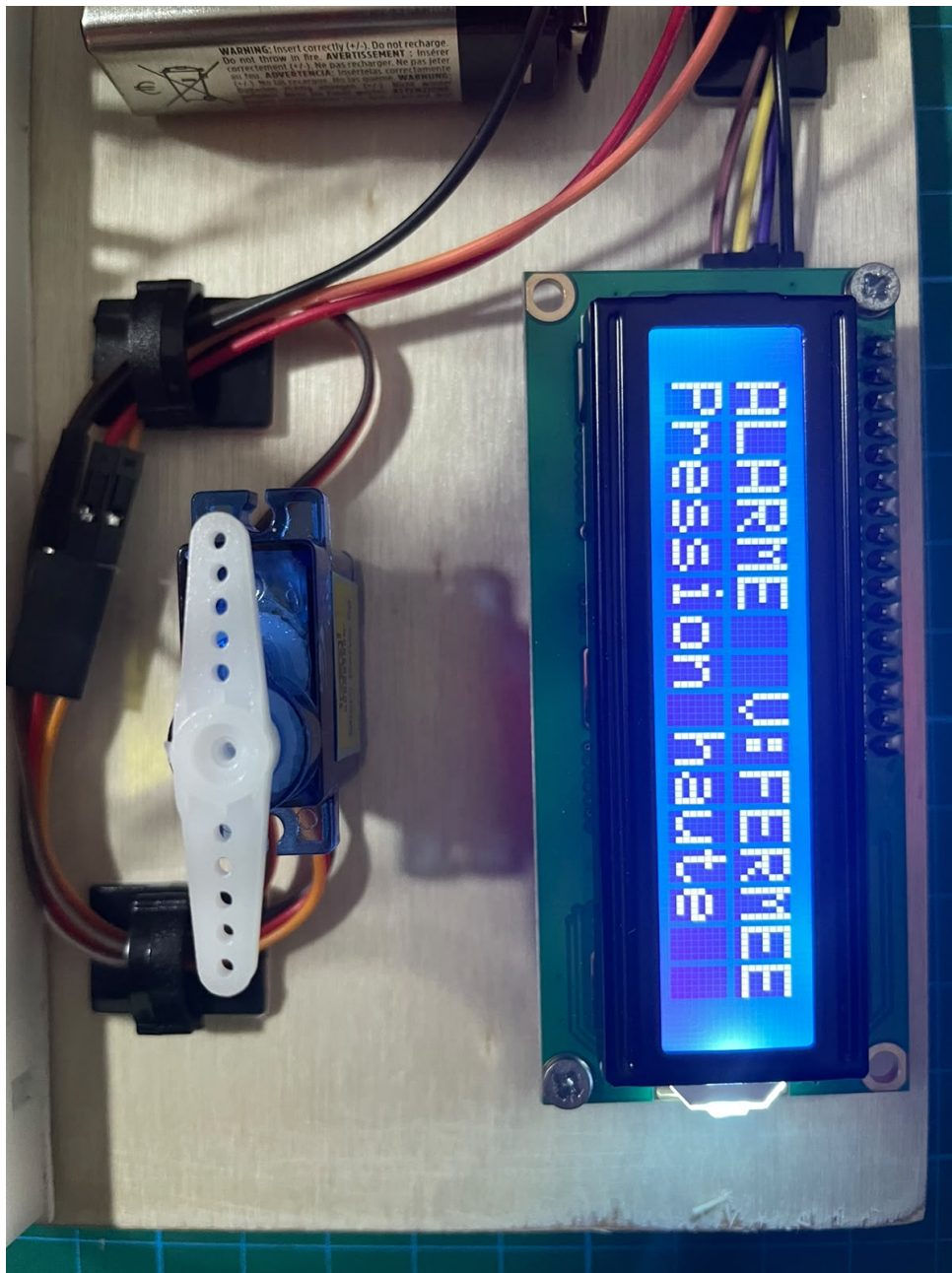


FIGURE 9.2 – Affichage LCD en fonctionnement alarme(état vanne + lectures capteurs).

10. Limitations identifiées

TABLE 10.1 – Limitations du prototype et mitigations proposées.

Limitation	Impact	Mitigation / Production
Servo : pas de fail-safe natif	Vanne reste figée si coupure électrique	Actionneur à ressort de rappel en production
HW-038 : max 540/1023 (non 1023)	Plage de mesure réduite	Seuil calibré empiriquement à 370/1023
Pas de boîtier IP	Exposition à l'humidité	Boîtier IP65 recommandé en production
Pression simulée (potentiomètre)	Non représentatif d'une vraie pression	Capteur de pression 4–20mA en production
Thermistance NTC non marquée : B supposé	Précision absolue dégradée loin de 28°C	Calibration deux points (bain de glace) ou PT100 en production

11. Discussion

La comparaison montre que l'actionneur pneumatique est la meilleure solution pour une installation industrielle réelle, principalement en raison de la sécurité positive native. L'actionneur pneumatique avec ressort de rappel ferme la vanne automatiquement lors de toute panne, sans intervention logicielle. Il est important de souligner que la fermeture sur **coupure de courant** ne peut être assurée que mécaniquement : le code gère les trois défauts capteurs (température, pression, niveau) et le bouton d'urgence, mais en cas de perte d'alimentation le microcontrôleur s'arrête et ne commande plus rien. C'est précisément l'argument décisif en faveur d'un actionneur à rappel par ressort.

Pour le prototype étudiant, la solution électrique avec servomoteur SG90 est plus simple, moins coûteuse et directement commandable par Arduino. Elle permet de démontrer la logique FSM, la lecture multi-capteurs, le verrouillage d'alarme avec acquittement, et l'affichage temps réel.

La mesure de température est assurée par une thermistance NTC non marquée. Faute de fiche technique, le composant a été caractérisé expérimentalement : sa résistance a été mesurée à $8700\ \Omega$ à $28\ ^\circ\text{C}$, et le calcul est ancré sur cette mesure réelle plutôt que sur une valeur nominale supposée, ce qui annule l'incertitude sur la résistance nominale. Le coefficient $B = 3950$ a été adopté par défaut (valeur typique des thermistances $10\ \text{k}\Omega$), en attendant une calibration à deux points pour une précision accrue. Cette approche, bien que demandant un pont diviseur et un modèle Beta, fournit une mesure fiable dans la fenêtre $15\text{--}30\ ^\circ\text{C}$ requise.

Les seuils ont été déterminés empiriquement lors des essais : niveau d'eau à $370/1023$ (maximum observé $540/1023$ à immersion complète), distance « cuve pleine » à $5\ \text{cm}$, surpression à $2,5\ \text{bar}$. L'écran LCD 16×2 I2C (adresse $0x27$) affiche l'état du système, la position de la vanne et, le cas échéant, la cause précise du défaut.

Remarque : la « panne d'air comprimé » du cahier des charges ne concerne que la solution pneumatique. Le prototype étant électrique, ce mode de défaillance ne s'applique pas ; il est néanmoins traité nativement par l'actionneur pneumatique recommandé, dont le ressort de rappel ferme la vanne en cas de perte d'air comme de perte de courant.

12. Conclusion

Ce projet a permis d'étudier l'automatisation d'une vanne papillon dans une cuve de fermentation de vin. L'actionneur pneumatique quart de tour avec ressort de rappel est retenu comme solution industrielle recommandée, en raison de sa sécurité positive native. Pour le prototype, un servomoteur SG90 commandé par Arduino Uno simule le mouvement de la vanne.

Le système intègre plusieurs composants de mesure et d'affichage : niveau de liquide (HC-SR04), débordement (HW-038), pression simulée (potentiomètre $10k\Omega$), température (thermistance NTC), arrêt d'urgence manuel (bouton poussoir), indication d'état (LED verte/rouge, buzzer) et affichage temps réel (LCD 16×2 I2C). L'ensemble du système a été testé et validé sur prototype : la vanne suit son cycle temporisé, tout défaut provoque une fermeture verrouillée avec signalisation, et le retour au fonctionnement normal exige un acquittement manuel une fois la cause levée. Les seuils d'alarme ont été déterminés empiriquement lors des essais.

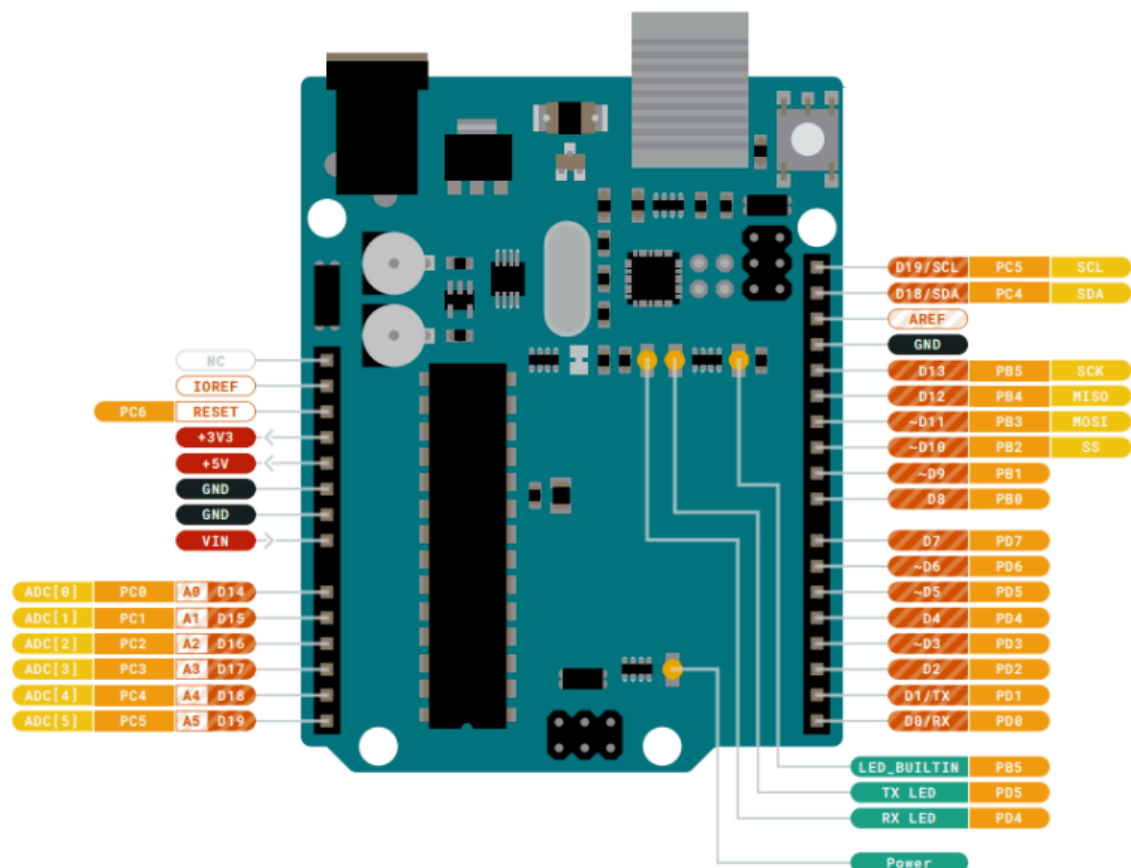
13. Améliorations possibles

- Remplacement de la thermistance NTC par un PT100 ou un thermocouple pour une meilleure précision en environnement industriel humide ;
- Calibration à deux points de la thermistance (bain de glace) pour déterminer le Beta réel ;
- Remplacement du servomoteur par un actionneur industriel pneumatique 24V à ressort de rappel ;
- Ajout d'un vrai capteur de pression 4–20mA à la place du potentiomètre ;
- Utilisation d'un boîtier étanche IP65 pour l'électronique ;
- Ajout d'une communication Wi-Fi ou Bluetooth pour supervision à distance ;
- Stockage des alarmes et historique des capteurs sur carte SD ;
- Remplacement de l'Arduino par un automate programmable industriel (API).

Appendix

- Documentation Arduino Uno — <https://docs.arduino.cc/hardware/uno-rev3>
- Thermistance NTC 10k Ω B3950 (type MF52, représentatif) — fiche fabricant
- Datasheet HC-SR04 — <https://cdn.sparkfun.com/datasheets/Sensors/Proximity/HCSR04.pdf>
- Datasheet servomoteur SG90 — https://handsontec.com/dataspecs/motor_fan/SG90-Servo.pdf
- Module I2C LCD 16 $\times$ 2 — https://www.handsontec.com/dataspecs/module/I2C_1602_LCD.pdf
- Datasheet HW-038 Water Level Sensor - https://curtocircuito.com.br/datasheet/sensor/nivel_de_agua_analogico.pdf
- Notes de cours sur les Actionneurs — Université Libanaise, Faculté de Génie, 2025/2026

Arduino Datasheet



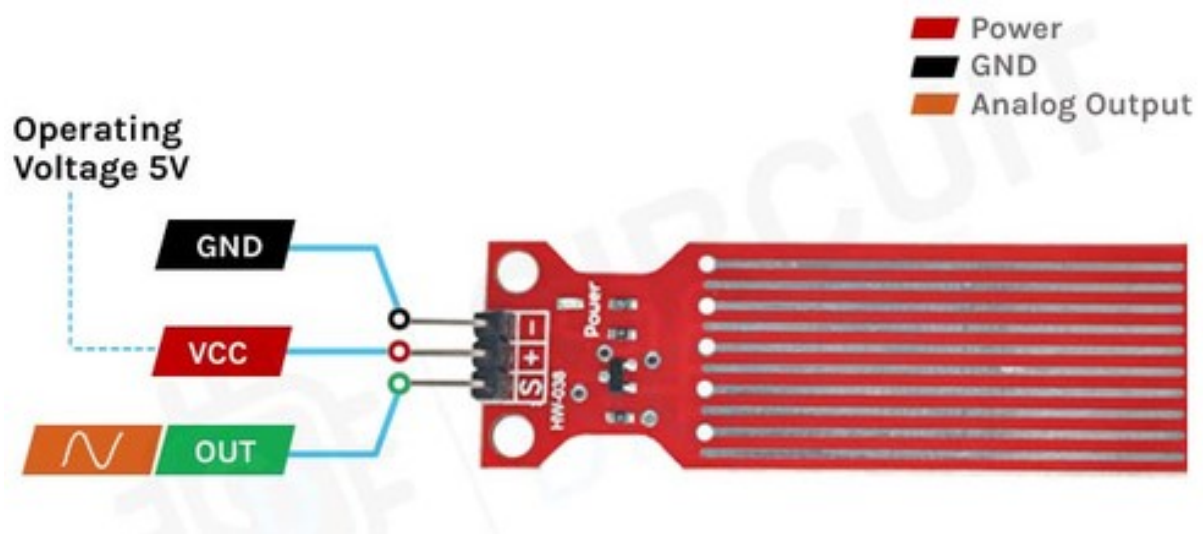
Ultrasonic Sensor Datasheet



Electric Parameter

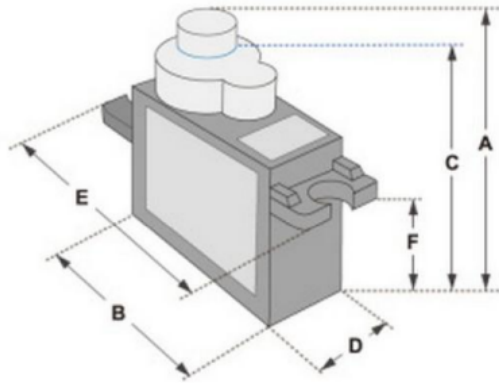
Working Voltage	DC 5 V
Working Current	15mA
Working Frequency	40Hz
Max Range	4m
Min Range	2cm
MeasuringAngle	15 degree
Trigger Input Signal	10uS TTL pulse
Echo Output Signal	Input TTL lever signal and the range in proportion
Dimension	45*20*15mm

Water Level Sensor Datasheet



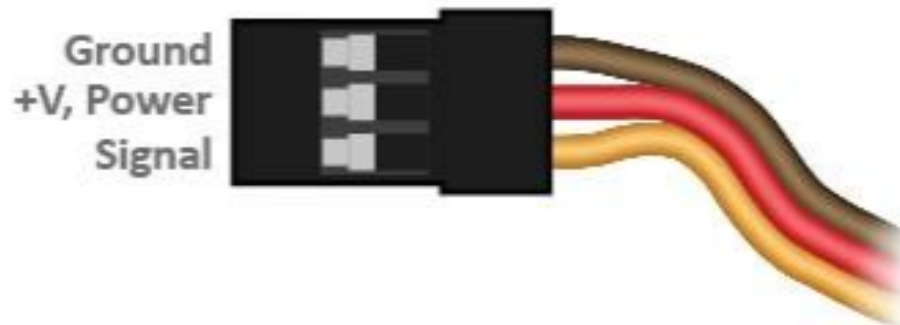
Servo-Motor Datasheet

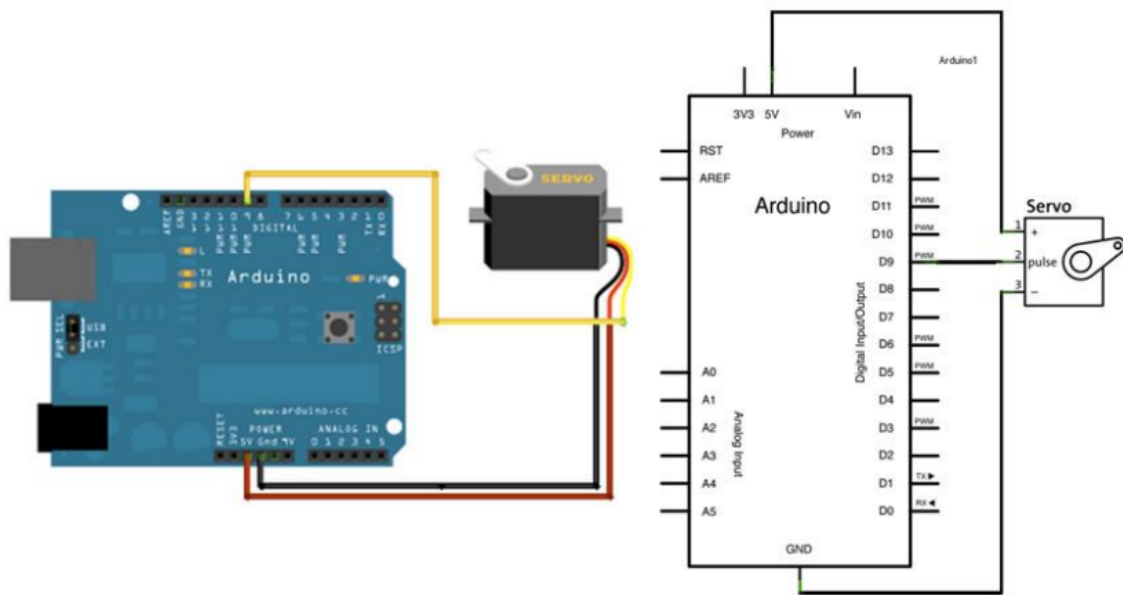
Mechanical Dimensions:



Dimensions & Specifications	
A (mm) :	32
B (mm) :	23
C (mm) :	28.5
D (mm) :	12
E (mm) :	32
F (mm) :	19.5
Speed (sec) :	0.1
Torque (kg-cm) :	2.5
Weight (g) :	14.7
Voltage :	4.8 - 6

CONNECTOR PINOUT:





LCD Datasheet

